Stratégie de tests logiciel / tests de produit

BLOC 1 — FONDATIONS

Séance 1 — Introduction à la stratégie de test

Objectifs

- Comprendre les enjeux du test
- Différence bug / défaut / incident
- Coût du bug

L'enjeu des tests logiciels

1. Garantir que le logiciel fait ce qu'on attend

Un test sert d'abord à répondre à une question simple : "Est-ce que mon code fonctionne comme prévu ?"

Mais attention :

- "fonctionne" ≠ "compile"
- "fonctionne" ≠ "pas de bug visible"

En réalité :Un test valide un **comportement attendu**

Exemple :

```
int divide(int a, int b)
```

Sans test :

- tu penses que ça marche

Avec test :

- tu vérifies :
  - 10 / 2 = 5
  - 10 / 0 → exception
  - -10 / 2 = -5

Le test **formalise les attentes**

2. Réduire le coût des bugs (enjeu majeur)

Loi fondamentale en informatique :Plus un bug est détecté tard, plus il coûte cher

Ordre de grandeur :

- Dev : 1€

- Recette : 10€
- Production : 100€ à 1000€

Exemple réel

- bug en prod :
    - perte d'argent
    - image de marque détruite
    - clients perdus

Cas typiques :

- paiement débité 2 fois
- impossibilité de commander
- fuite de données

3. Permettre d'évoluer sans casser (non-régression)

Le vrai enjeu des tests n'est pas seulement le présent, mais le futur.
"Est-ce que mon changement casse quelque chose ?"

Exemple

Tu modifies :

```
calculatePrice()
```

Sans tests :
tu casses une règle métier → personne ne voit
Avec tests :

- un test échoue immédiatement → sécurité

Les tests deviennent : un **filet de sécurité**

4. Accélérer le développement

Ça peut paraître contre-intuitif :
Écrire des tests fait gagner du temps

Pourquoi ?

- moins de bugs → moins de debug
- moins de régressions
- confiance pour refactorer

Sans tests :

- peur de modifier le code
- dette technique
- ralentissement progressif

5. Améliorer la qualité du code

Un effet souvent sous-estimé : Le test influence la façon de coder

Exemple :

Code difficile à tester :

```
new Service().doEverything();
```

pousse à :

- découper
- injecter dépendances
- clarifier responsabilités

Résultat :
meilleur design logiciel (clean code)

6. Documenter le comportement

Un test est une documentation vivante

Exemple :

```
@Test
 void should_apply_discount_when_user_is_premium()
```

Sans lire le code, tu comprends :

- règle métier
- comportement attendu

Les tests sont souvent **plus fiables que la doc écrite**

7. Sécuriser la production

Tests = réduction des risques :

- bugs critiques
- crashs
- indisponibilité

Couplé à :

- CI/CD
- déploiement automatique

→ permet : livrer souvent et en sécurité

8. Tester les situations extrêmes (robustesse)

Un bon système doit gérer :

- erreurs
- charge
- pannes

Types de tests associés :

- crash test
- test de charge
- chaos engineering

Exemple :

- DB down
- API externe lente
- mémoire saturée

9. Aligner les équipes (dev, QA, PO)

Les tests deviennent un langage commun

- PO → définit les règles
- QA → écrit scénarios
- Dev → implémente

Avec TDD / BDD : le test devient la **spécification**

Les tests servent à :

1. Vérifier le comportement
2. Réduire le coût des bugs
3. Éviter les régressions
4. Accélérer le développement
5. Améliorer le design
6. Documenter le code
7. Sécuriser la production
8. Tester les cas extrêmes

Différence entre défaut, bug et incident

1. Défaut

Définition :

Un défaut est une **erreur dans le code ou la conception**.
C'est le problème *à la source*.

Exemple

```
int divide(int a, int b) {
      return a / b;
```

Problème :

- aucun contrôle si b == 0

Le **défaut** :
absence de gestion de la division par zéro

Important :

- Le défaut peut exister **sans être encore visible**
- Il peut rester caché longtemps

2. Bug

Définition :
Un bug est la **manifestation concrète d'un défaut**.
C'est quand le problème devient observable dans le logiciel.

Exemple

Tu exécutes :

```
divide(10, 0);
```

Résultat :

- crash / exception

Le **bug** :
le programme plante quand on divise par zéro

Donc :

- Défaut = cause
- Bug = effet visible dans le code

3. Incident

Définition :
Un incident est un **problème observé en production** qui impacte un utilisateur ou un service.
C'est le niveau **business / utilisateur**

Exemple

Contexte :

- appli e-commerce

Étape 1 — Défaut
Le développeur oublie de gérer un cas :

```
if (user.isPremium()) {
      applyDiscount();
 }
```

Défaut :

- pas de gestion du cas “user null”

Étape 2 — Bug
Dans l’application :

- certaines requêtes plantent avec une NullPointerException

Bug :

- erreur visible dans l’app

Étape 3 — Incident
En production :

- les utilisateurs ne peuvent plus passer commande

Incident :

- perte de ventes
- impact client réel

Résumé simple

Terme	Niveau	Description
Défaut	Code	Erreur dans le code
Bug	Technique	Comportement incorrect observable
Incident	Utilisateur	Impact réel en production

exemple

Jeu vidéo

- Défaut : mauvaise formule de calcul des dégâts
- Bug : les dégâts sont incohérents dans le jeu
- Incident : les joueurs quittent le jeu / mauvaise note

Paiement

- Défaut : erreur dans le calcul du montant
- Bug : le prix affiché est faux
- Incident : client débité trop → plainte

Point important

Tous les défauts ne deviennent pas des bugs
Tous les bugs ne deviennent pas des incidents. Mais : chaque incident vient d'un bug, lui-même causé par un défaut

Coût d'un bug

Définition :
Le coût d'un bug correspond à **l'effort, l'argent et les ressources nécessaires pour le détecter, le corriger et gérer ses conséquences**. Plus un bug est détecté tard dans le cycle de développement ou en production, plus son coût augmente.

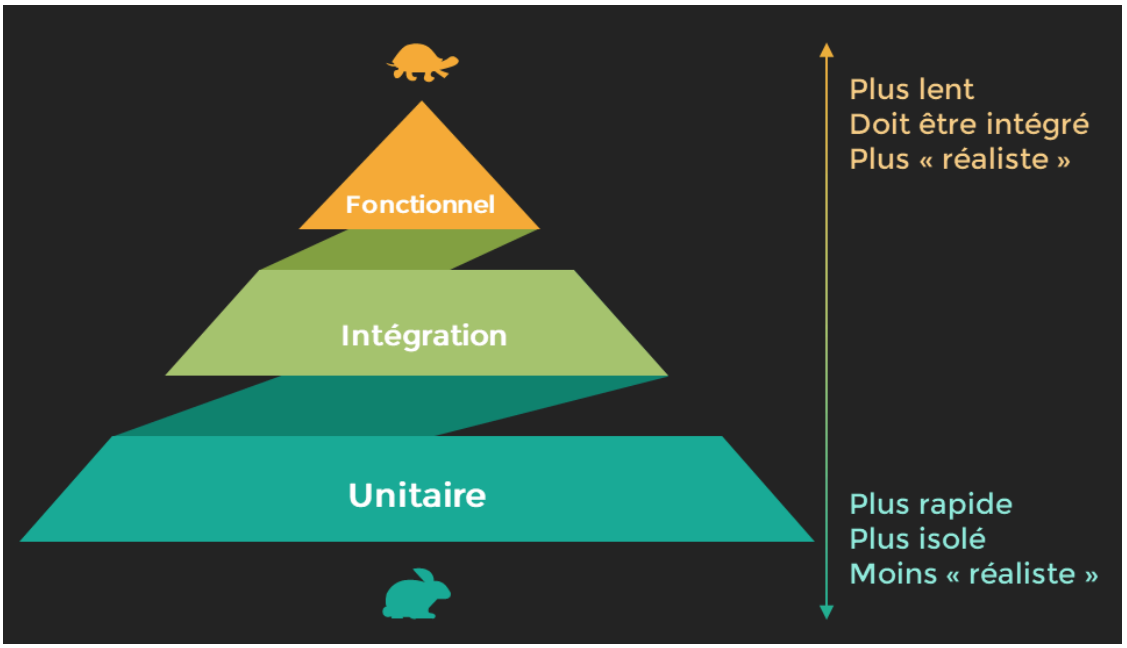
Exemple

Imaginons une application bancaire :

- Pendant le développement, un développeur oublie de vérifier si un montant est négatif.
- Si ce bug est détecté immédiatement en test unitaire, il suffit d'ajouter une condition. Coût : de quelques minutes à une heure de travail.
- Si le bug n'est découvert qu'après le déploiement en production et qu'il cause un mauvais débit sur des comptes clients, il faudra rembourser les clients, notifier les régulateurs, faire une mise à jour urgente et gérer les plaintes. Coût : plusieurs milliers d'euros et impact sur la réputation de la banque.

Théorie

- Pyramide des tests



Types de tests logiciels

Test unitaire (Unit Test)

Définition :
Un test unitaire vérifie le **fonctionnement d'une seule unité de code**, généralement une fonction ou une méthode, isolée de tout le reste.

Exemple :

- Méthode `calculateDiscount(user)`
- Test : si l'utilisateur est premium, le rabais est bien appliqué.

Objectif : vérifier que chaque "brique" fonctionne correctement avant de l'assembler.

Test d'intégration (Integration Test)

Définition :
Le test d'intégration vérifie que **plusieurs modules ou composants fonctionnent correctement ensemble**.

Exemple :

- Une API qui récupère un utilisateur depuis la base de données et calcule son panier.
- Test : vérifier que la récupération depuis la DB + le calcul du panier fonctionne correctement.

Objectif : détecter les problèmes d'interactions entre composants.

Test de bout en bout (End-to-End / E2E)

Définition :
Le test E2E vérifie **le fonctionnement complet d'un scénario réel** depuis le début jusqu'à la fin, comme le verrait un utilisateur.
Exemple :

- Scénario : un client crée un compte, ajoute un produit au panier, passe la commande et reçoit un email de confirmation.
- Test : automatiser ce parcours complet pour s'assurer qu'aucune étape ne casse.

Objectif : s'assurer que le logiciel fonctionne globalement comme attendu.

Test de non-régression (Regression Test)

Définition :
Le test de non-régression vérifie qu'**une modification du code n'a pas cassé des fonctionnalités existantes**.

Exemple :

- Après avoir ajouté une nouvelle méthode de paiement, on relance les tests existants sur le panier et la commande pour s'assurer que rien d'autre n'a été impacté.

Objectif : maintenir la stabilité du logiciel au fil des évolutions.

Activité par groupe

Tester un distributeur de billets

Stack distributeur de billets

Interface utilisateur

La première couche est l'interface avec l'utilisateur. C'est tout ce que le client voit et manipule : l'écran, le clavier ou le tactile. C'est ici que l'utilisateur saisit son code PIN, choisit un montant, consulte son solde ou voit les messages d'erreur.

Cette partie peut sembler simple, mais elle est critique car elle guide l'utilisateur. Une mauvaise gestion des messages ou des erreurs peut rendre l'expérience incompréhensible ou frustrante.

Logique métier

Derrière l'interface se trouve le cœur du système : la logique métier du distributeur. C'est elle qui prend les décisions.

Elle vérifie si le code PIN est correct, contrôle le solde du compte, applique les règles comme les plafonds de retrait, et gère des cas particuliers comme le blocage de la carte après plusieurs tentatives échouées.

C'est ici que se trouvent la majorité des règles fonctionnelles, et donc une grande partie des risques de bugs.

Système bancaire externe

Le distributeur ne fonctionne pas seul. Il doit communiquer avec le système de la banque via une API.

Cette communication permet de vérifier les informations du compte, autoriser ou refuser un retrait, et débiter le montant. Cette couche introduit des contraintes supplémentaires comme la latence réseau, les erreurs de communication ou l'indisponibilité du service.

C'est une source majeure de problèmes en conditions réelles, et elle est souvent sous-estimée par les étudiants.

Matériel

Enfin, il ne faut pas oublier la partie physique. Le distributeur contient un lecteur de carte, un système de distribution des billets, une imprimante pour les tickets et différents capteurs.

Cette couche introduit des problèmes très concrets : carte illisible, billets bloqués, manque de cash, imprimante en panne. Ce sont des situations fréquentes dans la réalité, mais rarement prises en compte spontanément.

Vision d'ensemble

On peut résumer le fonctionnement comme une chaîne :

L'utilisateur interagit avec l'interface, qui transmet les actions à la logique métier. Cette logique appelle le système bancaire pour valider les opérations, puis le matériel exécute l'action physique, comme distribuer les billets.

Chaque maillon de cette chaîne peut être testé, individuellement ou dans son ensemble.

Consignes :

- chaque groupe propose une stratégie de test
- restitution + débat

Séance 2 — Qualité logicielle & stratégie

- stratégie de test (quoi tester ? quand ?)

  Une stratégie de test consiste à définir **ce que l'on teste et à quel moment**, afin de garantir la qualité du logiciel tout en optimisant l'effort. L'objectif n'est pas de tout tester, mais de tester de manière pertinente, en fonction des risques et de la valeur métier.

Quoi tester ?

On ne teste pas tout de la même manière. Une bonne stratégie repose sur une priorisation intelligente.

Les fonctionnalités critiques

Ce sont celles qui ont un impact direct sur l'utilisateur ou le business. Si elles échouent, le produit perd immédiatement sa valeur.

Exemples :

- paiement
- authentification
- validation de commande

Les règles métier

Elles définissent le comportement attendu du système et sont souvent sources d'erreurs.

Exemples :

- application d'une remise pour un utilisateur premium
- plafond de retrait
- validation d'un panier uniquement si le paiement est accepté

Les cas limites et erreurs

Un logiciel doit être robuste face aux situations imprévues. Ces cas sont souvent oubliés mais critiques en production.

Exemples :

- données invalides ou nulles
- division par zéro
- service externe indisponible

Les interactions entre composants

Un système est composé de plusieurs briques qui doivent fonctionner ensemble. Même si chaque partie fonctionne seule, leur interaction peut poser problème.

Exemples :

- API avec base de données
- communication entre services
- appel à un système externe

Quand tester ?

Tester ne doit pas être une étape finale. Plus un bug est détecté tôt, plus il est facile et rapide à corriger.

Pendant le développement

Les tests unitaires sont utilisés pour valider les briques de base. Ils permettent un retour rapide et sécurisent le code dès sa création.

Lors de l'assemblage du système

Les tests d'intégration vérifient que les composants fonctionnent correctement ensemble. Ils interviennent lorsque les différentes parties commencent à interagir.

Avant livraison

Les tests end-to-end permettent de valider des scénarios complets du point de vue utilisateur. Ils assurent que le système fonctionne globalement.

À chaque modification

Les tests de non-régression garantissent que les évolutions n'ont pas cassé des fonctionnalités existantes. Ils sont essentiels pour maintenir la stabilité dans le temps.

- Test produit vs test technique

Dans une stratégie de test, il est important de distinguer deux approches complémentaires : le test technique et le test produit. Les deux sont nécessaires, mais ils ne répondent pas aux mêmes objectifs.

Test technique

Le test technique vise à vérifier que le code fonctionne correctement d'un point de vue informatique. Il s'intéresse à la logique interne, aux algorithmes et aux interactions entre composants.

On cherche ici à répondre à la question : est-ce que le système fonctionne correctement d'un point de vue technique ?

Ce que l'on teste

- méthodes et fonctions
- règles de calcul
- interactions entre services
- accès aux bases de données

Exemples

- vérifier qu'une méthode retourne le bon résultat
- tester qu'un service récupère correctement des données
- valider qu'une API répond avec le bon format

Test produit

Le test produit vise à vérifier que le logiciel répond au besoin utilisateur et aux objectifs métier. Il ne s'intéresse pas à la manière dont le code est écrit, mais au résultat global.

On cherche ici à répondre à la question :
est-ce que le produit fait ce que l'utilisateur attend ?

Ce que l'on teste

- parcours utilisateur
- règles fonctionnelles

- expérience globale
- cohérence du comportement

Exemples

- un utilisateur peut créer un compte et se connecter
- un client peut passer une commande de bout en bout
- le système applique correctement une remise

Différence clé

Le test technique regarde le système de l'intérieur, alors que le test produit regarde le système de l'extérieur.

- Test technique : centré sur le code
- Test produit : centré sur l'utilisateur

Exemple

Prenons un paiement en ligne.

Un test technique va vérifier que :

- la méthode de calcul du montant est correcte
- l'appel à l'API de paiement fonctionne
- les données sont bien enregistrées

Un test produit va vérifier que :

- un utilisateur peut payer sans erreur
- le bon montant est débité
- la commande est validée et confirmée

Pourquoi les deux sont indispensables

Se limiter aux tests techniques peut donner un faux sentiment de sécurité : le code fonctionne, mais le produit peut être inutilisable ou incorrect du point de vue métier.

À l'inverse, ne faire que du test produit rend difficile la détection précise des erreurs et leur correction.

Une bonne stratégie combine donc les deux :

- les tests techniques assurent la solidité du code
- les tests produit garantissent la valeur pour l'utilisateur

- Notion de couverture

Parfait, c'est un point important — souvent mal compris par les étudiants.
Je te fais une version claire, pédagogique, avec un bon équilibre.

Notion de couverture

La couverture de test mesure **la part du code qui est exécutée lors des tests**. Elle permet d'avoir une idée du niveau de vérification du logiciel, mais elle ne garantit pas à elle seule la qualité des tests.

Définition

La couverture correspond au **pourcentage de code parcouru par les tests**.

Par exemple, si 80 % des lignes de code sont exécutées pendant les tests, on parle de 80 % de couverture.

Types de couverture

Il existe plusieurs manières de mesurer la couverture, chacune apportant une vision différente.

Couverture de lignes

Elle mesure le nombre de lignes de code exécutées.

Exemple :
Si une méthode contient 10 lignes et que 8 sont exécutées pendant les tests, la couverture est de 80 %.

Couverture de branches

Elle vérifie que tous les chemins possibles sont testés.

Exemple :

```
if (user.isPremium()) {
      applyDiscount();
 } else {
      applyStandardPrice();
 }
```

Il faut tester les deux cas :

- utilisateur premium
- utilisateur non premium

Couverture de conditions

Elle vérifie les différentes combinaisons logiques.

Exemple :

```
if (user != null && user.isActive())
```

Il faut tester :

- user null
- user non null mais inactif
- user actif

À quoi sert la couverture

La couverture permet de :

- détecter les zones non testées
- identifier les parties critiques oubliées
- améliorer progressivement la qualité des tests

C'est un indicateur utile pour piloter les efforts de test.

Limites de la couverture

Une couverture élevée ne signifie pas que les tests sont bons.

On peut atteindre 100 % de couverture avec des tests inutiles qui ne vérifient rien de pertinent.

Exemple :
Un test qui appelle une méthode sans vérifier le résultat augmente la couverture, mais n'apporte aucune garantie.

Exemple concret

Méthode :

```
int max(int a, int b) {
      return a > b ? a : b;
 }
```

Si tu testes seulement :

```
max(5, 3)
```

Tu as une couverture partielle, car le cas inverse n'est pas testé.

Pour une bonne couverture, il faut aussi tester :

```
max(3, 5)
```

Bonne pratique

La couverture doit être utilisée comme un **outil de pilotage**, pas comme un objectif absolu.

Il vaut mieux :

- 70 % de couverture avec des tests pertinents
  que
- 100 % de couverture avec des tests inutiles

Une bonne stratégie consiste à combiner couverture et réflexion sur les cas réellement importants à tester.

Atelier

analyser une app (ex: Uber)

- quels tests ?
- quels risques ?

Séance 3 — Introduction aux tests unitaires (Java)

TD

- JUnit 5
- assertions
- structure AAA (Arrange Act Assert)

Exercice :

- tester une classe Calculator
- tester cas limites

Séance 4 — Bonnes pratiques de test

Théorie

- tests lisibles

Les tests unitaires ne servent pas uniquement à vérifier le fonctionnement du code : ils sont aussi **un moyen de communication**. Un test lisible doit permettre à n'importe quel développeur de comprendre rapidement **ce que l'on teste, pourquoi et comment**. La lisibilité est donc une qualité fondamentale des tests, au même titre que leur précision ou leur exhaustivité.

Pourquoi la lisibilité est importante

Un test illisible est presque inutile. Même s'il détecte correctement les bugs, il devient difficile à maintenir, à étendre ou à corriger lorsque le code évolue. Dans une équipe, les tests lisibles permettent à tous les membres de comprendre les intentions initiales, de détecter rapidement les cas oubliés et de contribuer à la qualité globale du logiciel.

La lisibilité réduit également le risque de **faux positifs ou négatifs** : un test compréhensible aide à interpréter correctement les résultats et à identifier l'origine d'un échec.

Bonnes pratiques pour écrire des tests lisibles

1. Utiliser des noms explicites

Le nom du test doit décrire clairement ce qu'il teste et dans quelles conditions.
Exemples :

- Mauvais : `test1()`
- Correct : `addition_de_deux_entiers_positifs_retourne_resultat_correct()`
- Correct et plus synthétique : `shouldReturnSumWhenAddingTwoPositiveNumbers()`

2. Suivre la structure AAA

La structure **Arrange – Act – Assert** rend le test lisible et compréhensible.

- **Arrange** : préparer les données et le contexte
- **Act** : exécuter la méthode ou la fonctionnalité à tester
- **Assert** : vérifier que le résultat correspond aux attentes

Exemple :
// Arrange

```
int a = 5;
int b = 3;
```

// Act

```
int result = calculator.add(a, b);
```

// Assert

```
assertEquals(8, result);
```

3. Limiter la complexité

Un test doit rester simple et ciblé. Il est préférable d'écrire plusieurs tests courts plutôt qu'un test long qui mélange plusieurs fonctionnalités.

- Un test = une intention

- Éviter les boucles, conditions complexes ou multiples assertions dans un même test si elles testent des choses différentes

4. Éviter le code inutile

Supprimer tout ce qui n'apporte pas d'information sur le test : variables non utilisées, calculs complexes ou commentaires redondants. Le test doit parler par lui-même.

5. Utiliser des constantes et des objets explicites

Des valeurs magiques ou abstraites rendent le test moins lisible. Préférer des noms explicites pour les données :

```
int montantCommande = 100;
```

```
 int remiseUtilisateurPremium = 10;
```

6. Commenter uniquement ce qui est nécessaire

Un test bien nommé et bien structuré n'a pas besoin de commentaires pour expliquer chaque ligne. Les commentaires doivent seulement expliquer **le contexte métier ou le raisonnement complexe**, pas la mécanique du code.

Exemple concret

Mauvais exemple :

```
@Test
 void test1() {
      int r = calc.add(5,3);
      assertEquals(8, r);
 }
```

Bon exemple :

```
@Test
 void shouldReturnSumWhenAddingTwoPositiveNumbers() {
      // Arrange
      int a = 5;
      int b = 3;
// Act
      int result = calculator.add(a, b);
// Assert
```

```
    assertEquals(8, result);
}
```

La différence est claire : dans le second test, **le nom, la structure et les variables rendent le test immédiatement compréhensible**, même pour un développeur qui découvre le projet.

- test isolé

Un **test isolé** est un test qui vérifie le comportement d'une unité de code **de manière indépendante** de ses dépendances externes. L'objectif est de s'assurer que la fonctionnalité testée fonctionne correctement **quelle que soit l'environnement ou l'état des autres composants**.

Les tests isolés sont essentiels pour garantir **la fiabilité, la rapidité et la maintenabilité** des tests unitaires.

Pourquoi isoler un test

Sans isolation, un test peut dépendre de l'état d'un autre composant, d'une base de données, d'une API externe ou d'un service tiers. Dans ce cas :

- un échec peut provenir d'un autre module et non du code testé
- le test devient instable et difficile à reproduire
- l'exécution peut être lente ou coûteuse

Isoler le test permet de **détecter rapidement l'origine exacte d'un problème**, de rendre les tests plus robustes et de faciliter le développement dirigé par les tests (TDD).

Comment isoler un test

1. Utiliser des mocks ou stubs

Les **mocks** remplacent les dépendances par des versions simulées qui renvoient des réponses prédéfinies. Cela permet de tester le composant en **contrôlant entièrement son environnement**.

Exemple avec Mockito :

```
when(serviceExterne.getData()).thenReturn("valeur simulée");
```

2. Ne tester qu'une seule unité

Un test isolé doit se concentrer sur **une seule classe ou méthode**.
Si plusieurs unités sont testées en même temps, il devient difficile de savoir quelle partie a causé un échec.

3. Éviter les dépendances réseau ou base de données

Pour rester rapide et fiable, un test isolé **ne doit pas accéder à une base de données réelle, à un fichier ou à Internet**.
On peut utiliser :

- des mocks pour remplacer les appels
- des bases de données en mémoire pour simuler les données

Exemples

Test non isolé (mauvais exemple)

@Test

```
 void testGetUserName() {
      User user = userService.getUserFromDatabase(1);
      assertEquals("Alice", user.getName());
 }
```

- Dépend d'une base de données réelle
- Si la DB change ou n'est pas accessible, le test échoue même si getName() fonctionne

Test isolé

```
@Test
 void shouldReturnUserName() {
      // Arrange
      UserRepository mockRepo = mock(UserRepository.class);
      when(mockRepo.findById(1)).thenReturn(new User(1, "Alice"));
      UserService service = new UserService(mockRepo);
// Act
      String name = service.getUserName(1);
// Assert
      assertEquals("Alice", name);
 }
```

- Dépendance à la base simulée par un mock
- Le test est **rapide, fiable et indépendant**

Bonnes pratiques pour les tests isolés

- Un test = une seule responsabilité
- Utiliser les mocks pour toutes les dépendances externes
- Préparer le contexte avec des données simples et explicites
- Éviter de mélanger plusieurs unités ou composants dans le même test
- Nommer le test de façon descriptive pour indiquer clairement ce qui est testé

- anti-patterns

Dans le développement dirigé par les tests (TDD) et la stratégie de tests, il est important de connaître non seulement ce qu'il faut faire, mais aussi **ce qu'il faut éviter**. Les **anti-patterns de tests** sont des pratiques qui semblent correctes au départ mais qui **réduisent la qualité, la lisibilité ou la fiabilité des tests**.

Les reconnaître permet d'écrire des tests plus efficaces, maintenables et compréhensibles.

1. Test trop complexe

Un test trop long ou qui mélange plusieurs intentions est difficile à comprendre et à maintenir.

Problèmes :

- difficile de comprendre ce qui est réellement testé
- un échec ne montre pas clairement quelle partie du code pose problème
- maintenance coûteuse lors d'évolution du code

Exemple :

```
@Test
 void testEverything() {
      int sum = calculator.add(2,3);
      int prod = calculator.multiply(sum,5);
      int max = calculator.max(prod, 50);
      assertEquals(50, max);
 }
```

Ce test mélange plusieurs opérations. Si l'assert échoue, on ne sait pas immédiatement quelle opération est en cause.

Bonnes pratiques :

- un test = une seule intention
- séparer les cas pour chaque méthode ou fonctionnalité

2. Test dépendant de l'environnement

Un test qui dépend de la base de données réelle, de fichiers ou d'un service externe devient **instable et lent**.

Exemple :

```
@Test
 void testGetUserFromDatabase() {
      User user = userService.getUserFromDatabase(1); // dépend de la DB
      assertEquals("Alice", user.getName());
 }
```

- Si la base change ou n'est pas disponible, le test échoue même si le code est correct.

Solution :

- utiliser des mocks ou stubs pour isoler le test
- privilégier les tests unitaires rapides et indépendants

3. Test fragile (flaky test)

Un test fragile échoue **de manière aléatoire** selon l'ordre d'exécution, l'environnement ou le timing.

Exemple :

- test qui dépend de l'heure système
- test qui lit des fichiers temporaires
- test qui dépend d'un thread ou d'un service externe

Conséquence :

- perte de confiance dans la suite de tests
- temps perdu à relancer les tests et analyser les échecs

Bonnes pratiques :

- éviter les dépendances temporelles ou externes
- utiliser des données fixes et contrôlées

4. Test redondant ou inutile

Les tests qui répètent exactement ce que d'autres tests vérifient ou qui ne testent rien de concret **alourdissent la suite de tests** sans apporter de valeur.

Exemple :

```
@Test
 void testAdditionDoesNotThrow() {
      calculator.add(2, 3);
 }
```

- Ici, aucune assertion, le test ne vérifie rien d'utile.

Solution :

- chaque test doit vérifier un résultat attendu ou un comportement spécifique
- supprimer les tests redondants

5. Test non lisible

Un test mal nommé ou mal structuré devient rapidement **illisible pour les autres développeurs**.

Exemple :

```
@Test
 void t1() {
      int r = calculator.add(5,3);
      assertEquals(8, r);
 }
```

- Que teste exactement ce test ? On ne sait pas sans regarder le code.

Solution :

- nom explicite : `shouldReturnSumWhenAddingTwoPositiveNumbers()`
- utiliser la structure AAA (Arrange, Act, Assert)

6. Test qui casse facilement lors de refactoring

Un test trop couplé à l'implémentation interne du code (par exemple tester des variables privées ou la structure interne) **rend le refactoring difficile**.

Exemple :

- tester la liste interne d'un objet au lieu du comportement observable
- vérifier des détails d'implémentation inutiles

Solution :

- tester le comportement observable
- privilégier les assertions sur les résultats ou effets visibles

TD

code volontairement mauvais à corriger

Objectifs

- Identifier les **anti-patterns de tests** : tests trop complexes, dépendants de l'environnement, redondants, illisibles ou non isolés
- Refactorer des tests pour les rendre **lisibles et fiables**
- Appliquer les bonnes pratiques de TDD : structure AAA, assertions claires, tests isolés

BLOC 2 — TECHNIQUE SPRINGBOOT

Séance 5 — Tests Spring Boot

TD

- Comprendre la différence entre **tests d'intégration** (@SpringBootTest) et **tests ciblés sur un controller** (@WebMvcTest)
- Tester un **controller REST** dans un projet Spring Boot
- Appliquer les bonnes pratiques : tests isolés pour le controller, assertions sur le contenu JSON, statuts HTTP

```
@SpringBootTest
```

  - Permet de **charger tout le contexte Spring**, utile pour tester des intégrations complexes.
  - Plus lourd que @WebMvcTest.

```
@WebMvcTest
```

  - Ne charge **que le controller** et les beans liés au Web MVC
  - Utilise MockMvc pour tester les endpoints rapidement et isolément

1. Étudier les différences entre `@SpringBootTest` et `@WebMvcTest`.
2. Ajouter un **nouveau test pour la gestion des utilisateurs non trouvés** (status 404).
3. Ajouter un **test de validation** pour POST : vérifier qu'un utilisateur sans nom renvoie une erreur.
4. Exécuter les tests avec mvn test ou depuis l'IDE pour vérifier leur bon fonctionnement.

Séance 6 — Mocking

Comprendre **l'injection de dépendances** et le rôle d'un repository dans un service

- Utiliser **Mockito** pour isoler le service et ne pas toucher à la base de données
- Écrire des tests **unitaires lisibles et isolés** pour la logique métier

- Mockito
- Dépendances

1. Identifier **où Mockito est utilisé** et quel est le rôle de @Mock et @InjectMocks.
2. Ajouter un **test pour le cas où l'utilisateur n'existe pas** (retour null).
3. Ajouter un **test de comportement** : vérifier que save n'est jamais appelé si l'utilisateur est null.
4. Discuter de la différence entre **tests unitaires avec mock** et **tests d'intégration**.

TD

tester un service avec mock repository

Séance 7 — Tests d'intégration

TD

- Comprendre la différence entre **tests unitaires avec Mockito** et **tests d'intégration réels**
- Tester le **service et le repository** en interaction avec une base en mémoire H2
- Vérifier **toutes les opérations CRUD** : Create, Read, Update, Delete
- Appliquer les bonnes pratiques : tests isolés, rollback automatique

1. Exécuter tous les tests pour vérifier le CRUD complet.
2. Ajouter un **test pour récupérer un utilisateur inexistant** et vérifier que le service retourne null.
3. Ajouter un **test pour vérifier le comportement de deleteUser sur un ID inexistant**.
4. Discuter de la différence entre ce test **d'intégration réel** et le test avec **Mockito du service** (isolation vs intégration).
5. Observer que chaque test est **rollbacké** grâce à @Transactional, donc la base H2 reste propre.

- H2
- test DB

CRUD complet

Séance 8 — Tests API avancés

TD

- Comprendre comment tester des **endpoints REST** dans Spring Boot
- Apprendre à utiliser **MockMvc** pour des tests unitaires d'API (mock du contexte web)
- Apprendre à utiliser **RestAssured** pour des tests plus proches d'intégration
- Vérifier les codes HTTP, les JSON renvoyés et la logique métier

1. Exécuter **MockMvc** pour comprendre le **test isolé du controller**.
2. Exécuter **RestAssured** pour comprendre le **test intégration avec serveur lancé**.
3. Ajouter un test pour **GET /api/users/{id}** inexistant → vérifier qu'on obtient null ou 404.
4. Ajouter un test pour **vérifier les codes HTTP** en cas d'erreur.
5. Discuter des différences **MockMvc vs RestAssured** :
    - MockMvc = test rapide, pas besoin de serveur
    - RestAssured = test plus proche du réel, serveur lancé

- MockMvc

- RestAssured

tester endpoints REST

Séance 9 — Tests E2E

- notion de bout en bout

- Comprendre ce qu'est un **test E2E** : tester le flux complet d'une fonctionnalité du début à la fin
- Combiner plusieurs endpoints pour **vérifier un scénario utilisateur**
- Vérifier le **comportement réel** de l'application (base, service, controller, endpoints REST)
- Appliquer les bonnes pratiques : tests isolés, rollback après chaque test

1. Exécuter le test E2E pour vérifier le scénario **Créer user** → **Login** → **Vérifier liste**
2. Ajouter un test pour **login échoué** avec un mauvais mot de passe → vérifier qu'on retourne null ou code HTTP approprié
3. Ajouter un test pour **login avec utilisateur inexistant**
4. Discuter de la différence entre :
    - Tests unitaires (isolés, mock)
    - Tests d'intégration (repository + service)
    - Tests E2E (flux complet, plusieurs endpoints enchaînés)

TD

scénario :

- créer user
- login
- action

BLOC 3 — TDD

Séance 10 — Introduction au TDD

- cycle Red / Green / Refactor

Atelier live coding

Objectifs

- Comprendre le cycle **Red** → **Green** → **Refactor**
- Apprendre à **écrire les tests AVANT le code**
- Découvrir l'intérêt du TDD : design, sécurité, lisibilité
- Pratiquer via un **live coding guidé**

Le cycle TDD

RED

On écrit un test qui échoue
le besoin n'est pas encore implémenté

GREEN

On écrit le minimum de code pour faire passer le test
pas de perfection, juste faire passer

REFACTOR

On améliore le code sans casser les tests
clean code, duplication, lisibilité

Philosophie

Le test décrit le besoin

- Le code répond au besoin
- Le test protège contre la régression

Atelier live coding

Problème à résoudre

Implémenter une classe StringUtils avec une méthode :
int countVowels(String input)
Compter le nombre de voyelles (a, e, i, o, u)

Étape 1 — RED

Tu écris uniquement le test :

```
import org.junit.jupiter.api.Test;
 import static org.junit.jupiter.api.Assertions.*;
class StringUtilsTest {
@Test
      void shouldCountVowelsInWord() {
      StringUtils utils = new StringUtils();
      int result = utils.countVowels("hello");
      assertEquals(2, result);
      }
 }
```

Ça ne compile pas → NORMAL (TDD)

Étape 2 — GREEN

Implémentation minimale :

```
public class StringUtils {
public int countVowels(String input) {
      return 2; // hack volontaire
      }
 }
```

Le test passe

Code nul mais acceptable en TDD

Étape 3 — RED

Ajouter un nouveau test :

```
@Test
 void shouldReturnZeroWhenNoVowels() {
      StringUtils utils = new StringUtils();
      assertEquals(0, utils.countVowels("rhythm"));
 }
```

Test échoue

Étape 4 — GREEN

Amélioration minimale :

```
public int countVowels(String input) {
      int count = 0;
      for (char c : input.toCharArray()) {
      if ("aeiou".contains(String.valueOf(c))) {
            count++;
      }
      }
      return count;
 }
```

Les tests passent

Étape 5 — REFACTOR

Amélioration :

```
public int countVowels(String input) {
      if (input == null) return 0;
int count = 0;
      String vowels = "aeiou";
for (char c : input.toLowerCase().toCharArray()) {
      if (vowels.indexOf(c) != -1) {
            count++;
      }
      }
return count;
 }
```

TD :

Exercice 1

Ajouter les tests suivants AVANT code :

- chaîne vide → 0
- null → 0
- majuscules → fonctionne

Exercice 2

Implémenter une nouvelle méthode en TDD :

boolean isPalindrome(String input)

Exemples :

- “radar” → true
- “hello” → false

Exercice 3

Implémenter :

int add(String numbers)

Format :

- “” → 0
- “1” → 1
- “1,2” → 3

- “1,2,3” → 6

(String Calculator kata simplifié)

Séance 11 — TDD en groupe

Activité

Kata (FizzBuzz)
Organisation :

- pair programming
- rotation toutes les 10 min

Objectifs

- Appliquer le cycle **Red / Green / Refactor**
- Travailler en **pair programming (driver / navigator)**
- Apprendre à **construire une solution progressivement**
- Écrire des **tests lisibles et incrémentaux**

Organisation

- Travail en binôme
- 1 driver (code), 1 navigator (réfléchit)
- **Rotation toutes les 10 minutes**

Règles du kata FizzBuzz

Pour un nombre :

- divisible par 3 → "Fizz"
- divisible par 5 → "Buzz"
- divisible par 3 et 5 → "FizzBuzz"
- sinon → le nombre en String

Séance 12 — TDD avancé

TD — Construire une feature complète en TDD

Objectifs

- Appliquer le **TDD de bout en bout** sur une feature métier
- Structurer une feature : **Controller → Service → Repository**
- Écrire des **tests avant le code** à chaque niveau
- Travailler en **petits incréments (Red → Green → Refactor)**
- Produire du code **testé, lisible et maintenable**

Contexte

Vous devez développer une fonctionnalité simple d'un système utilisateur :

Un utilisateur peut :

- s'inscrire
- se connecter
- être refusé si ses données sont invalides

Feature à implémenter

1. Inscription utilisateur

Créer un endpoint :
POST /api/users/register
Comportement attendu :

- Créer un utilisateur avec username et password
- Refuse si :
    - username déjà existant
    - password trop court (< 6 caractères)

2. Login utilisateur

Créer un endpoint :
POST /api/users/login
Comportement attendu :

- Retourne l'utilisateur si les credentials sont corrects
- Retourne une erreur sinon

Contraintes TDD

Règles OBLIGATOIRES :

- Interdiction d'écrire du code métier sans test
- Chaque fonctionnalité doit suivre :
    - RED → test qui échoue
    - GREEN → code minimal
    - REFACTOR → amélioration

Organisation

- Travail en **binôme (pair programming)**
- Rotation toutes les **10 minutes**
- Un codeur, un reviewer

Structure attendue

Vous devez créer :

- User (entité)
- UserRepository
- UserService
- UserController

Aide :

Entité

```
@Entity
 @Table(name = "users")
 public class User {
@Id
      private String username;
private String password;
// getters / setters
 }
```

Repository

```
public interface UserRepository extends JpaRepository<User, String> {
      Optional<User> findByUsername(String username);
 }
```

Service (à compléter en TDD)

```
@Service
 public class UserService {
private final UserRepository repository;
public UserService(UserRepository repository) {
      this.repository = repository;
      }
```

```
public User register(String username, String password) {
      // TODO TDD
      return null;
      }
public User login(String username, String password) {
      // TODO TDD
      return null;
      }
 }
```

Travail demandé (par étapes TDD)

Étape 1 — Inscription simple

Écrire un test :

- `register("alice", "password")` → utilisateur créé

Étape 2 — Username déjà existant

Test :

```
register("alice", "password")
register("alice", "password2")
```
→ erreur attendue

Étape 3 — Mot de passe invalide

Test :

- password < 6 → erreur

Étape 4 — Login OK

Test :

utilisateur existant + bon password → OK

Étape 5 — Login KO

Tests :

- mauvais password → erreur
- user inexistant → erreur

Tests attendus

```
@Test
 void shouldRegisterUser() {
      User user = userService.register("alice", "password");
      assertEquals("alice", user.getUsername());
 }
```

Extension :

Controller REST

Créer :

```
@RestController
 @RequestMapping("/api/users")
 public class UserController {
      // register + login
 }
```

Tester avec :

- MockMvc OU RestAssured

Critères de réussite

- Tests écrits AVANT le code
- Couverture des cas :
  - OK
  - erreurs
- Code lisible
- Pas de duplication
- Respect du cycle TDD

BLOC 4 — QUALITÉ PRODUIT & CHAOS

Séance 13 — Crash test & résilience

- test de charge

  Un **test de charge** est un type de test de performance qui consiste à **évaluer le comportement d'un système lorsqu'il est soumis à un volume élevé d'utilisateurs ou de requêtes simultanées**, proche ou supérieur à son usage réel.

  L'objectif est de vérifier que l'application reste **fonctionnelle, stable et performante** sous charge, sans dégradation excessive (temps de réponse, erreurs, saturation).

Concrètement, on simule par exemple :

- des centaines ou milliers d'utilisateurs connectés en même temps
- des appels API massifs
- des pics de trafic

On mesure ensuite :

- le **temps de réponse**
- le **taux d'erreur**
- l'**utilisation des ressources** (CPU, mémoire)
- le **point de saturation**

Exemple : tester une API de paiement avec 1000 requêtes simultanées pour vérifier qu'elle répond toujours en moins de 300 ms sans erreur.

- test de robustesse

  Un **test de robustesse** est un type de test qui vise à vérifier comment un système **réagit face à des conditions anormales, imprévues ou extrêmes**, au-delà de son usage nominal.

  L'objectif n'est pas seulement de voir si le système fonctionne, mais surtout de s'assurer qu'il **ne plante pas**, qu'il **résiste aux erreurs** et qu'il **se comporte de manière contrôlée** (messages d'erreur propres, pas de crash, pas de corruption de données).

  Concrètement, on teste par exemple :

- des **entrées invalides** (null, formats incorrects, données incohérentes)
- des **volumes extrêmes** (très grandes tailles de données)
- des **conditions dégradées** (service indisponible, timeout, perte réseau)
- des comportements inattendus des utilisateurs

On observe :

- la **stabilité du système**
- la **gestion des erreurs**
- la capacité à **continuer de fonctionner partiellement**
- l'absence de crash ou de fuite mémoire

Exemple : envoyer un JSON invalide à une API et vérifier qu'elle retourne une erreur 400 propre au lieu de planter avec une exception interne.

- fault injection

  Un **test de fault injection** (injection de fautes) consiste à **introduire volontairement des pannes ou des erreurs dans un système** afin d'observer son comportement et de vérifier sa **résilience**.

Contrairement au test de robustesse (où on envoie des entrées invalides), ici on **simule des défaillances internes ou externes**.

L'objectif est de valider que le système :

- **résiste aux pannes**
- **se dégrade proprement**
- **se rétablit correctement**
- respecte ses mécanismes de **tolérance aux pannes**

Concrètement, on injecte par exemple :

- une **base de données indisponible**
- un **service externe qui répond lentement ou en erreur**
- une **coupure réseau**
- une **exception forcée dans le code**
- une **latence artificielle**

On observe ensuite :

- le comportement des **fallbacks**
- la gestion des **timeouts et retries**
- l'impact sur les utilisateurs
- la capacité de **recovery**

Exemple : couper l'accès à la base de données pendant un test et vérifier que l'API retourne une erreur contrôlée (503) sans crasher, et qu'elle reprend normalement une fois la base rétablie.

Atelier — Comment casser votre application ? (Revolut)

Objectif :
Se mettre dans la peau d'un utilisateur malveillant ou d'un système en panne pour identifier les failles d'une application.

Consigne :
En groupe, imaginez et proposez un maximum de scénarios permettant de faire échouer une application (fonctionnellement ou techniquement).

Pistes à explorer :

- entrées invalides ou extrêmes
- surcharge (beaucoup d'utilisateurs ou requêtes)
- indisponibilité d'un service externe
- incohérences métier (double paiement, solde négatif…)
- latence ou coupure réseau

Exemple :

Comme dans une app type banque (ex : Revolut), que se passe-t-il si :

- deux paiements sont lancés en même temps ?
- le service de paiement tombe en panne après débit ?
- une requête est rejouée plusieurs fois ?

Restitution :
Chaque groupe présente ses scénarios et explique :

- ce qui casse
- pourquoi
- comment le système devrait réagir

Séance 14 — Chaos engineering

- principes du chaos engineering

  Les **principes du chaos engineering** sont un ensemble de règles visant à tester la **résilience d'un système** en provoquant volontairement des pannes de manière contrôlée.

  ## Définition

  Le chaos engineering consiste à **introduire des perturbations dans un système pour vérifier qu'il continue de fonctionner correctement et de manière maîtrisée**.

  ## Principes clés

  1. Définir le comportement normal
  On identifie les métriques de référence : temps de réponse, taux d'erreur, disponibilité.

  2. Formuler une hypothèse
  On exprime ce qui devrait se passer en cas de panne.
  Ex : "Si le service X tombe, alors le système reste disponible avec un fallback."

  3. Injecter des perturbations
  On simule des pannes réelles : coupure réseau, latence, service indisponible.

  4. Observer et mesurer
  On analyse l'impact sur le système et les utilisateurs.

  5. Limiter le risque
  Les tests doivent être progressifs, contrôlés et réversibles.

STACK TECHNIQUE

- Java 17+
- Spring Boot
- JUnit 5
- Mockito
- H2
- MockMvc
- Maven/Gradle